
RESEARCH TRACK - STUDY GUIDE

BRSR

Mathematical foundations of a metadata-driven BRSR data platform: canonical normalisation, safe Decimal calculation, ratio-of-sums consolidation, statistical anomaly detection, and checksummed reporting.

Person 4 - Mathematics & Research
Documentation - 2026

BRSR PLATFORM - PERSON 4 GUIDE

Table of Contents

1. How to Use This Document	2
2. The Problem Statement in Research Terms	2
3. The System at a Glance	3
4. The Framework as a Mathematical Object	3
5. Unit Normalisation: A Canonical Map	4
6. The Safe Calculation Engine	5
7. Consolidation: Why $\Sigma_{\text{num}}/\Sigma_{\text{den}}$ Is the Correct Mathematics	5
8. The Validation Engine: Ten Rule Classes	6
9. Statistical Anomaly Detection: IQR and Z-Score	7
10. The Workflow, Freeze, and Year-over-Year Continuity	8
11. Immutability, Audit, and the Ledger Model	8
12. Delivering the Explanation: Talk Plan and Q&A; Bank	9
13. Source Map and Constants Appendix	10
14. The One-Minute Close	12

1. How to Use This Document

This guide is your complete preparation pack for the **research and explanation track** of the BRSR (Business Responsibility and Sustainability Reporting) platform project. Your role on the team is distinct from the three technical roles: you own the **problem statement (PS)**, **the mathematical formulation of every mechanism in the system**, and **the research-grade justification** of the design decisions. When the panel asks "why is this statistically sound?" or "prove that your consolidated numbers are correct", that is your moment.

The document is organised so that each section answers three questions in order: **(a) what is the mechanism**, **(b) what is its formal/mathematical statement**, and **(c) why was it designed that way** (the research argument). Every number, threshold, and formula quoted here is taken directly from the implementation (file references are given in Section 13), so anything you present is verifiable against the running code.

Your deliverables

1) Methodology chapter with formal notation for every mechanism; 2) a constants/threshold appendix; 3) defense-ready answers to the Q&A; bank (Section 12); 4) traceability from each implemented feature back to the problem statement. Read Sections 2-11 to learn, Sections 12-13 to prepare for delivery.

2. The Problem Statement in Research Terms

BRSR is the disclosure framework mandated by SEBI (Securities and Exchange Board of India) for the top listed entities. It requires companies to report quantitative and qualitative performance across nine principles of the National Guidelines on Responsible Business Conduct - from ethics and governance (P1) through employee wellbeing (P3), environmental stewardship (P6), and consumer responsibility (P9). The reporting burden is real: the framework used here defines **57 metrics** across Sections A (general disclosures, 9), B (management process, 3) and C (principle-wise performance, 45), of which **16 are BRSR Core indicators** - the subset subject to assessment and assurance.

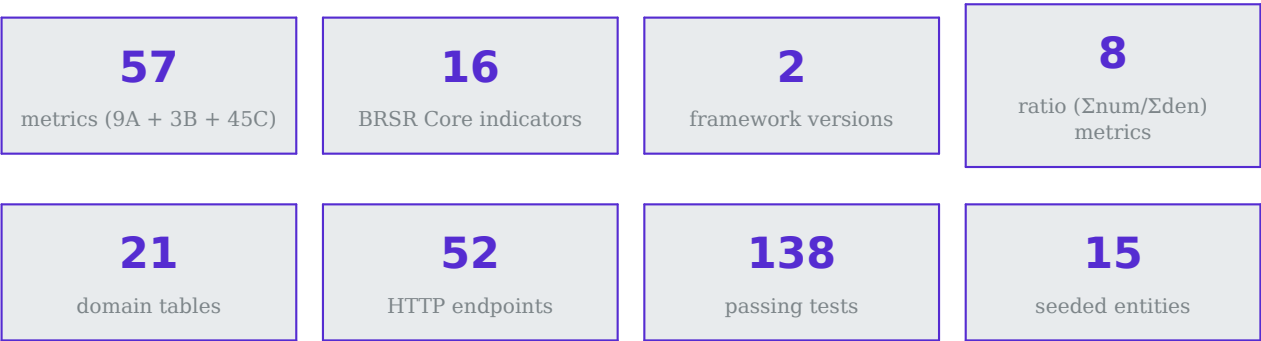
The research problem the platform attacks has three horns, and you should present it exactly this way:

- **Integrity of aggregates.** Group-level numbers are computed from plant-level numbers. A single non-audited spreadsheet edit, a silently averaged percentage, or a unit mix-up (kWh reported as MWh) corrupts the disclosed figure. The PS demands *provable* correctness of every aggregation, not best-effort correctness.
- **Accountability of provenance.** A disclosed number must be traceable backwards: which plant entered it, who approved it, which document (evidence) supports it, under which version of the metric definition. Sustainability reporting without lineage is not assurable.
- **Continuity across framework versions.** BRSR definitions change year over year (one metric in the seed data was redefined between FY2024-25 and FY2025-26). Comparing this

year to last year requires an explicit, machine-checked notion of what "the same metric" means.

Formally, the system treats reporting as a **directed dataflow with integrity guarantees**: raw observations flow through normalisation, validation, calculation, consolidation, and finally freeze into a checksummed snapshot. Each stage is a mathematical function with stated preconditions, and the invariant is that *no stage may silently transform or drop information* - errors are surfaced, never smoothed over. This "fail loudly" principle is the single most important research claim to internalise: everywhere the implementation could have guessed (division by zero, unknown unit, missing peer data), it raises instead.

3. The System at a Glance



The platform is a modular monolith: FastAPI + PostgreSQL + Redis + MinIO + Celery on the backend, Next.js 15 on the frontend, deployed with docker-compose. Fifteen entities form a recursive hierarchy (group, two subsidiaries, four business units, six plants, two projects) - this hierarchy is what consolidation walks over. Eleven users cover the role spectrum from ADMIN to data owners at each plant. The pipeline below is the map for everything that follows:

```
Entry / Import / AI suggestion → Normalisation → Validation (10 rule classes + statistical sweep) → Calculation (formulas) → Review workflow → Consolidation (bottom-up,  $\Sigma\text{num}/\Sigma\text{den}$ ) → Period lock → Snapshot (SHA-256) → PDF report
```

Note the direction of trust: everything downstream of review consumes only values with status APPROVED or LOCKED. This restriction is stated in the code and is one of the seven invariants (Section 11).

4. The Framework as a Mathematical Object

The deepest design decision in the platform: **the reporting framework itself is data, not code**. Metrics, units, formulas, and validation rules are rows in database tables (metadata), so adapting to a new BRSR cycle requires no code change. Mathematically, a framework version V is a structure:

$$V = (M, F, R, L)$$

- **M** - the metric catalogue: 57 tuples (code, section, principle, data_type, unit_family, canonical_unit, allowed_units, aggregation_semantics, num, den, ...). Data types are {numeric, qualitative, categorical, boolean}; aggregation semantics are {SUM, WEIGHTED_AVERAGE, RATIO_RECALCULATION, DERIVED_METRIC, DIRECT_VALUE}.
- **F** - versioned formulas. The seed defines three: total energy (grid renewable + grid non-renewable), Scope 1 GHG (diesel litres $\times$ emission factor $\div$ 1000, with constant DIESEL_EF = 2.68 kg CO₂e/litre), and total GHG (Scope 1 + Scope 2). Each formula version snapshots its input codes and constants as JSONB, so a computed value can always be reproduced bit-for-bit.
- **R** - validation rules (Section 8): 13 seeded instances drawn from 10 deterministic classes plus the statistical sweep.
- **L** - lineage: cross-version metric mappings of type {DIRECT, RENAMED, REDEFINED, SPLIT}. The seed contains one REDEFINED mapping (water used $\rightarrow$ water withdrawal), which is exactly the case that must be flagged "compare with care" in trend analysis (Section 10).

Database-level guarantees

The schema enforces the mathematics with CHECK constraints: a metric marked RATIO_RECALCULATION must declare both numerator and denominator; only Section C metrics may carry a principle; a ratio metric cannot be its own numerator or denominator; a numeric metric must have a canonical unit that is a member of its allowed units. Malformed metadata is rejected at insert time - the framework cannot drift into inconsistency.

5. Unit Normalisation: A Canonical Map

Different plants measure the same quantity in different units - one reports energy in kWh, another in MWh, a third in GJ. Comparison and summation across a heterogeneous set is only meaningful in a single canonical unit per *unit family*. Normalisation is a function:

$$v: (\text{value}, \text{unit}) \rightarrow (\text{value} \times f(\text{unit}), \text{canonical_unit}), \quad f(\text{unit}) > 0$$

implemented as a static factor table - no floating point, no runtime discovery. The canonical plan: energy $\rightarrow$ MWh (kWh = 0.001, GJ = 1/3.6), water $\rightarrow$ kL (litre = 0.001, ML = 1000), waste $\rightarrow$ tonne (kg = 0.001), emissions $\rightarrow$ tCO₂e (kg = 0.001), currency $\rightarrow$ INR crore (INR = 10⁻⁷, lakh = 0.01), fuel volume $\rightarrow$ litre (kL = 1000). Normalisation runs at every numeric save, so the normalised value column is always present and always canonical.

Research argument (say this in the viva)

An unknown unit, an unknown family, or a missing canonical unit raises UnitConversionError - the value is never silently treated as zero or passed through raw. Combined with exact Decimal arithmetic (Section 6), the composition of normalisation with any downstream aggregation is well-defined and order-independent, which is precisely the property a Σ -based consolidation needs.

6. The Safe Calculation Engine

Derived metrics and formulas are evaluated by a **recursive-descent expression evaluator** - never Python eval. The grammar is the classic arithmetic core:

```
expr := term (("+" | "-") term)* | term := factor (("*" | "/" ) factor)* | factor :=  
NUMBER | IDENT | "(" expr ")" | "-" factor
```

Three properties matter for the research narrative:

- **Exact decimal arithmetic.** All arithmetic runs in Decimal with precision 28, and results are quantised to 10^{-8} with ROUND_HALF_EVEN. Binary floating point cannot represent 0.1 exactly; currency and emission figures demand exactness and a stated rounding rule, which auditors can re-verify digit for digit.
- **Failure is total, not partial.** Division by zero raises CalculationError. An unknown identifier raises through the resolver: a missing input *never* becomes zero. A formula over incomplete data therefore produces no value at all, rather than a plausible-looking wrong one.
- **Reproducibility.** Every computed value stores its formula id, formula version, and a JSONB snapshot of the exact inputs used. Recomputation is idempotent: if the result, formula version, and inputs are unchanged, no new version is written. Editing any numeric input triggers recomputation of dependents.

7. Consolidation: Why Σnum/Σden Is the Correct Mathematics

Consolidation walks the entity hierarchy bottom-up (leaves first) and combines child values into parent values according to each metric’s aggregation semantics. The centrepiece is the treatment of ratio metrics such as LTIFR (lost-time injuries per million man-hours), attrition rate, renewable share, GHG intensity, and water intensity - eight metrics in total.

The central result (memorise this)

For a ratio metric $r = N/D$ defined over entities $i = 1..n$, the group-level ratio is computed as $r_{\text{group}} = (\sum N_i) / (\sum D_i)$ - the ratio of sums - and **never** as the average of the individual ratios $(1/n) \sum r_i$. Percentages are scaled: a percent-family metric multiplies num/den by 100 after the division.

A worked example makes the argument concrete. Suppose two plants report LTIFR:

Plant	Lost-time injuries (N)	Man-hours (D)	Ratio $r = N/D$
Plant A	2	400,000	5.0 per million man-hours
Plant B	1	100,000	10.0 per million man-hours

Averaging the ratios gives $(5.0 + 10.0)/2 = 7.5$. The ratio of sums gives $(2+1)/(400,000+100,000) \times 10^6 = 3/500,000 \times 10^6 = \mathbf{6.0}$. The average is wrong because it weights each plant equally regardless of exposure: Plant B, with a quarter of the man-hours, contributes half the weight. The ratio of sums is exactly the quantity the disclosure demands -

injuries per million man-hours *of the group* - and it equals the exposure-weighted mean of the plant ratios, $\Sigma(r_i \cdot D_i) / \Sigma D_i$. This is the single most quotable mathematical argument in the whole project: naive percentage averaging is a real, commonly observed disclosure error, and the platform rules it out by construction.

The remaining semantics: SUM adds child values (energy, water, waste, emissions, headcount). WEIGHTED_AVERAGE prefers ratio-recalculation when a numerator/denominator pair exists, with a documented fallback to the simple mean. DERIVED_METRIC and DIRECT_VALUE take the entity's own approved value. Two guards complete the picture: inputs are restricted to APPROVED/LOCKED values, and a zero denominator raises ConsolidationError rather than producing an undefined or infinite result. Every consolidated number persists a **ConsolidationTrace** - a read-model row recording the exact contributing value ids - so any group figure can be decomposed into its leaves. When a child value is later approved or changed, all ancestor traces are marked STALE with a reason, forcing recomputation before anything downstream trusts them.

8. The Validation Engine: Ten Rule Classes

Validation is a set of declarative rules stored as metadata, dispatched to exactly ten deterministic evaluator classes, plus a statistical sweep (Section 9). Rules fire at stated workflow stages (entry, submit, or review) and with severities BLOCKING, WARNING, or INFO. Only BLOCKING rules at the submit stage can refuse a submission; BLOCKING rules at review can refuse approval. The complete taxonomy:

Rule class	Formal condition	Seeded instances
REQUIRED_FIELD	value $\neq$ NULL required for BRSR Core metrics	VR-REQ-CORE (BLOCKING)
RANGE_VALIDATION	$\min \leq x \leq \max$ on normalised value	attrition 0-100; percent family 0-200
UNIT_VALIDATION	$\text{raw_unit} \in \text{metric.allowed_units}$	VR-UNIT (numeric metrics)
TYPE_VALIDATION	numeric metric = raw_value present	VR-TYPE
CROSS_FIELD_RECONCILIATION	$ x_a - x_b \leq \text{tolerance}$	payroll employees, tolerance 0
CROSS_SECTION_RECONCILIATION	same, across report sections	Section A vs C employees
YOY_VARIANCE	$ (x_t - x_{t-1}) / x_{t-1} \leq \tau$ (default 20%)	VR-YOY-ENERGY (WARNING)
DUPLICATE_DETECTION	$x_t = x_{t-1}$ (same value and unit)	VR-DUP (INFO)
EVIDENCE_COMPLETENESS	evidence_required $\Rightarrow$ &exists; evidence row	VR-EVIDENCE-CORE (BLOCKING)
LOGICAL_CONSISTENCY	discharge $> 0 \Rightarrow$ withdrawal > 0	VR-LOGIC-WATER (WARNING)

Findings persist as ValidationException rows. The lifecycle is meaningful: an OPEN exception whose condition later clears is auto-RESOLVED; an EXPLAINED exception (the data owner

filed a justification) survives. Every raise writes an EXCEPTION_RAISED audit event in the same transaction. A broken rule never crashes the pipeline - it degrades to an "unverified" finding, so validation failure is always visible rather than silent.

The YoY rule is a research talking point

Year-over-year variance compares the current value against the latest APPROVED/LOCKED normalised value from an earlier period, flags change beyond a threshold (default 20%), and skips the zero-denominator case. It is cheap detection of the classic disclosure errors: a restated baseline, a unit slip, or a stale copy of last year's number (which DUPLICATE_DETECTION catches from the other direction).

9. Statistical Anomaly Detection: IQR and Z-Score

Beyond per-entry rules, a cross-site sweep compares each plant against its **peer set**: for a numeric metric and period, the peer values are the latest normalised values with status APPROVED or LOCKED from entities of type PLANT or PROJECT. Two classical outlier tests run over this set, each with a minimum sample guard of 4 peers:

$$\text{IQR: } Q_1 = Q(0.25), Q_3 = Q(0.75), \text{ IQR} = Q_3 - Q_1; \\ \text{flag } x \text{ if } x < Q_1 - 1.5 \cdot \text{IQR} \text{ or } x > Q_3 + 1.5 \cdot \text{IQR}$$

$$\text{Z-score: } z = (x - \mu) / \sigma \quad (\text{population } \sigma, \text{ peer set includes } x); \quad \text{flag } |z| > 2.0$$

Quantiles use linear interpolation at position $(n-1) \cdot q$ - the standard "Type 7" definition used by R and numpy, so results are reproducible against any statistical package. Constants: IQR multiplier **1.5** (Tukey's fence), z threshold **2.0** ($\approx 95.4\%$ coverage under normality), min peers **4**. Both tests are skipped when the peer set is too small or the dispersion is zero (IQR = 0 or $\sigma = 0$) - a deliberate refusal to manufacture outliers from insufficient evidence.

Worked example (use this in the presentation). Peer values for water withdrawal, in kL: 10,000 / 11,200 / 12,000 / 13,500 / 48,000.

- $Q(0.25)$ at position $(5-1) \times 0.25 = 1.0 \rightarrow 11,200$; $Q(0.75)$ at position $3.0 \rightarrow 13,500$. IQR = 2,300. Upper fence = $13,500 + 1.5 \times 2,300 = \mathbf{16,950}$. The value 48,000 is flagged.
- Z-score: $\mu = 18,940$, $\sigma \approx 14,750$ (population), $z \approx (48,000 - 18,940) / 14,750 \approx \mathbf{1.97}$ - just below the 2.0 threshold. This is the intended behaviour: the two tests measure different things. The IQR test is rank-based and robust (a single extreme value barely moves the quartiles), while the z-score is mean-based and sensitive to exactly the outlier it hunts - which is why the platform runs *both* and treats a flag from either as a WARNING, prompting explanation rather than blocking entry. The data owner must either correct the value or file a justification (e.g. a genuine one-off spill or a newly commissioned line).

The sweep is idempotent per rule + entity + period, reports the observed value rounded to 6 decimal places, and embeds the Q1-Q3 bounds or the z-score and peer mean in the exception message, so a reviewer sees the reasoning, not just the verdict.

10. The Workflow, Freeze, and Year-over-Year Continuity

The assignment lifecycle is a **finite state machine** - the transition table is a literal map in the code, and every change is authorised by server-side role checks:

```
NOT_STARTED → IN_PROGRESS → SUBMITTED → UNDER_REVIEW → {APPROVED |  
NEEDS_CORRECTION | REJECTED}  
NEEDS_CORRECTION → IN_PROGRESS; REJECTED → IN_PROGRESS; APPROVED  
→ LOCKED; LOCKED → (terminal)
```

Two gates: starting a review runs the full review-stage validation sweep; approving is refused while any unresolved BLOCKING exception exists. NEEDS_CORRECTION and REJECT require a non-empty comment - a rejection without a reason is not expressible. Approval triggers STALE-marking of upstream consolidation traces and an APPROVED notification to the data owner.

Freeze and integrity. Locking a period (ADMIN/ESG_MANAGER only) requires the period ended, every assignment APPROVED/LOCKED, and no unresolved BLOCKING exceptions. The lock forces all values to LOCKED and blocks edits everywhere. Reporting then takes exactly one immutable snapshot per period: a JSONB payload of every latest value (including normalised values, formula provenance, submitter, and evidence file names + SHA-256 digests), whose **SHA-256 checksum** is stored and printed in the PDF footer. The checksum makes tampering detectable: re-computing the digest over the payload reproduces the stored hash or exposes alteration. A second snapshot attempt returns 409. The PDF itself is rendered asynchronously (WeasyPrint via Celery, with retries) and stored in MinIO.

Trends with honest gaps. Multi-year trend charts resolve each historical point through lineage: same version → *direct*; a forward or reverse lineage mapping → *remapped* (with a "compare with care" note when the mapping is REDEFINED or SPLIT); no mapping → *gap*. The chart never draws a line between differently defined metrics - continuity is claimed only when the data model can justify it. This is a small but very presentable research position: *visual continuity must be earned by semantic continuity*.

11. Immutability, Audit, and the Ledger Model

The platform models reported values as an **append-only ledger**, not a mutable store. A correction to a value creates a new row (version n+1); no UPDATE path exists for an existing version, and edits are refused once a value is SUBMITTED, UNDER_REVIEW, APPROVED or LOCKED, or once the period is locked. Optimistic concurrency (expected_last_version) prevents two editors from silently clobbering each other. Each audit event is inserted in the *same database transaction* as the business write - if the audit insert fails, the business change rolls back too. The two facts that make this auditable for real:

- A **database trigger** (BEFORE UPDATE OR DELETE on the audit table) raises an exception on any mutation attempt - the append-only property is enforced below the application layer, so even a compromised app process cannot rewrite history without a

visible, separate act of schema surgery.

- Audit events carry actor, action (a 10-value enum), object, entity, metric code, old/new values as JSONB, reason, evidence reference, and request id - enough to reconstruct any disclosed number's biography.

Human-in-the-loop AI (research section 9 of the project)

The AI extraction module produces **suggestions only**, never data. A provider (deterministic local demo extractor, or an OpenAI-compatible LLM at temperature 0 with document text truncated to 6,000 characters and framed as data-only to resist prompt injection) returns per-field {value, unit, confidence}; the demo extractor assigns confidence 0.87 for body matches and 0.55 for filename fallbacks. The extract-only-what-is-present constraint is enforced in code: a null value or unit yields no suggestion. An accepted suggestion creates a value version authored by the data owner with status IN_PROGRESS - it can never be born SUBMITTED/APPROVED/LOCKED, so accountability for every disclosed figure remains with a human.

The seven invariants to recite as the system's mathematical constitution: (1) values are immutable versions; (2) audit events commit in the same transaction and are append-only by trigger; (3) all authorisation is server-side RBAC with entity scoping; (4) ratios consolidate as $\Sigma \text{num} / \Sigma \text{den}$; (5) AI output is a pending suggestion that only a human can act on; (6) reports render exclusively from checksummed locked snapshots; (7) rules and metrics are database metadata, not code.

12. Delivering the Explanation: Talk Plan and Q&A; Bank

A 10-minute research narrative that matches the demo the technical team will run:

Min	Content
0-2	Problem: BRSR / SEBI context, 57 metrics, the three horns - aggregate integrity, provenance, cross-version continuity.
2-4	Architecture as dataflow: entry → normalise → validate → compute → review → consolidate → freeze → checksummed report.
4-6	The $\Sigma \text{num} / \Sigma \text{den}$ argument with the two-plant LTIFR example (7.5 vs 6.0).
6-8	Statistical sweep: IQR fences and z-scores, the 48,000 kL worked example, why both tests.
8-10	Integrity guarantees: append-only ledger, same-transaction audit, SHA-256 snapshot, trends with honest gaps; close on the seven invariants.

Q&A; bank - rehearse these aloud

- **Why not average percentages across plants?** Exposure weighting: the group ratio is the exposure-weighted mean of plant ratios, $\Sigma(r \cdot D) / \Sigma D$; simple averaging weights plants equally and is a genuine disclosure error. Demo: 7.5 vs 6.0.

- **Why Decimal and not float?** Exactness and a stated rounding rule (quantise to 10^{-8} , ROUND_HALF_EVEN); auditors can recompute digit-for-digit. Binary floats cannot represent 0.1 and accumulate error.
- **Why is the IQR test run alongside the z-score?** Complementary error modes: IQR is robust to the outlier it detects (rank-based); the z-score uses the mean and variance that the outlier itself distorts. In the worked example IQR flags 48,000 while $z \approx 1.97$ stays under 2.0. Running both casts a wider, explainable net; flags are WARNINGS that demand explanation, not rejection.
- **Why min_peers = 4?** With fewer peers the quartiles/variance are statistically meaningless; the sweep declines to judge rather than manufacture outliers. Also skipped when dispersion is zero (all peers equal).
- **How do you know a PDF was not edited after generation?** The snapshot payload's SHA-256 is stored and printed in the report footer; recomputing the digest over the payload verifies or exposes tampering. Snapshots are one per period and immutable (409 on repeat).
- **What stops someone editing an approved value?** Three layers: status-based edit refusal, period lock, and the append-only version model (correction = new version $n+1$, with optimistic concurrency).
- **Can the AI enter data?** No. Suggestions are PENDING with per-field confidence; acceptance creates an IN_PROGRESS version authored by the human owner. Extract-only-what-is-present is enforced in code (null = no suggestion), and the LLM path is temperature 0 with data-only framing against prompt injection.
- **How do trends handle a redefined metric?** Lineage mapping types DIRECT / RENAMED / REDEFINED / SPLIT; unmapped comparisons degrade to explicit gaps, and redefinitions carry a "compare with care" note. Continuity is never implied semantically.
- **What if a validation rule itself is buggy?** Rules never crash the pipeline: a failing evaluator surfaces as an "unverified" finding. Exceptions auto-resolve when the condition clears; EXPLAINED ones persist with the owner's justification.
- **Why is the framework metadata, not code?** Annual BRSR cycles change metrics and definitions; versioned metadata (with lineage) lets a new cycle coexist with the old one and keeps history interpretable - plus DB CHECK constraints make malformed metadata unrepresentable.

13. Source Map and Constants Appendix

Every claim above is verifiable in the repository at these locations - cite them if challenged:

Topic	Implementation
Metric catalogue, rules, formulas seed	backend/app/framework/seeddata.py
Metadata invariants (CHECK-level logic)	backend/app/framework/service.py
Unit normalisation	backend/app/normalization/units.py
Safe evaluator	backend/app/calculation/evaluator.py

Topic	Implementation
Consolidation (Σ num/ Σ den, traces, staleness)	backend/app/consolidation/service.py
Rule classes and engine	backend/app/validation/engine.py
IQR / z-score statistics	backend/app/validation/statistics.py
Workflow transitions, submit gate	backend/app/collection/service.py
Value versioning model	backend/app/models/collection.py
Audit service + immutability trigger	backend/app/audit/service.py; migrations/.../a1b2c3d4e5f6_*
Lock, snapshot, checksum, PDF	backend/app/reporting/service.py
Trends continuity (direct/remapped/gap)	backend/app/trends/router.py
AI suggestions, human-in-the-loop	backend/app/ai/service.py
Data model overview	docs/DATABASE.md; docs/ARCHITECTURE.md

Constants appendix (quote these exactly)

Constant	Value	Where
Decimal precision / quantisation	28 digits; 10^{-8} , ROUND_HALF_EVEN	evaluator.py, units.py, consolidation
IQR multiplier (Tukey fence)	1.5	validation/statistics.py
Z-score threshold	2.0	validation/statistics.py
Minimum peer count	4	validation/statistics.py
YoY variance default threshold	20%	validation config
Percent plausibility range	0 - 200	seeded rule VR-RANGE-PERCENT
Attrition range	0 - 100	seeded rule VR-RANGE-ATTRITION
Diesel emission factor	2.68 kg CO ₂ e / litre	formula F-SCOPE1 constant
Demo extractor confidence	0.87 body / 0.55 filename	ai/service.py
LLM prompt text truncation	6,000 characters	ai/service.py
Evidence size cap	25 MB	upload limit
JWT lifetimes	60 min access / 7 days refresh	auth config
Reminder schedule	7 days before due / 3-day escalation, hourly scan	notifications config
Peer set statuses	APPROVED or LOCKED only	validation/statistics.py, consolidation

Final preparation checklist

Rehearse the LTIFR and water-withdrawal worked examples until you can do the arithmetic aloud. Recite the seven invariants. Walk the pipeline diagram once without notes.

Cross-check two code files of your choice against this document so you trust it. Then present - you are the person who turns a working system into a defensible one.

14. The One-Minute Close

If the panel remembers one thing from you, make it this: **the platform converts a disclosure obligation into a chain of well-defined functions, each with stated preconditions, exact arithmetic, and an enforced refusal to guess.** Percentages consolidate as ratios of sums, never averages. Outliers are judged by a robust rank test alongside a mean-based one, against a minimum peer evidence set. Every value is an immutable ledger entry with a same-transaction audit event; every report is a checksummed freeze of approved data; every trend line is drawn only where the data model can defend semantic continuity. The AI accelerates entry but never owns a number. That combination - exactness, provenance, and honest gaps - is what makes a sustainability disclosure *assurable*, and it is the research contribution of this project.

Practise the close twice: once in forty-five seconds (the paragraph above, spoken), once in three minutes (add the LTIFR example and the water-withdrawal IQR example). Own the mathematics, and the demo will feel inevitable rather than impressive.